PROGRAMACIÓN III

Trabajo Práctico Integrador: TypeScript & Autenticación

OBJETIVO GENERAL

Evolucionar la aplicación dinámica "Food Store" hacia un sistema con **Autenticación y Roles**. El alumno deberá implementar un flujo de seguridad que proteja el contenido según el tipo de usuario, sustituyendo la carga abierta de datos por un acceso restringido mediante TypeScript y localStorage.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Tipado Fuerte (TS)	Uso de Interfaces para asegurar que el usuario y sus roles (admin o client) sigan una estructura rígida y sin errores.
Persistencia (Local)	Uso de localStorage para simular una base de datos de usuarios y mantener la sesión activa del navegador.
Autenticación	Proceso de verificación de credenciales (Email y Password) comparándolos contra el almacenamiento local.
Autorización (Roles)	Lógica que decide si un usuario puede acceder a una ruta específica (ej. /admin/) basándose en su rol.

Caso Práctico

Actualmente, el repositorio base https://github.com/chiro45/proteger_rutas tiene una estructura de carpetas que debes respetar:

- **/pages**: Contenedores de las vistas (`auth`, `admin`, `client`).
- **/utils**: Lógica de navegación y verificación de sesión.
- **/types**: Definición de contratos de datos (`IUser`, `Rol`).

PASO 1: Registro de Usuarios (`src/pages/auth/registro/`)

Debes transformar el formulario estático en un sistema de captación de datos.

- Modificar el HTML para incluir: Email, contraseña y quitar el selector de rol.
- En el archivo `.ts`, capturar los datos y guardarlos en un **Array de Objetos** dentro del `localStorage` bajo la clave `"users"`.

PASO 2: Login y Gestión de Sesión (`src/pages/auth/login/`)

El login ya no debe ser una simulación; debe validar datos reales.

- Al intentar ingresar, buscar en el array de `"users"` si existe una coincidencia de email y contraseña.
- Si es correcto, guardar el objeto del usuario en la clave `"userData"` para iniciar la sesión.

PASO 3: Protección de Rutas (El "Guard")

Implementar la lógica para que las páginas no sean accesibles mediante URL si no hay sesión.

- **Centralización**: En `src/main.ts`, crear una función que intercepte la carga de la página.
- **Validación**: Si un usuario con rol `client` intenta entrar a una carpeta `/admin/`, debe ser redirigido automáticamente al login o a su zona permitida.

CONCLUSIONES ESPERADAS

Al finalizar, el alumno habrá logrado:

- Independizar la seguridad de la vista: Entender que la protección no es solo ocular, sino lógica.
- Dominar el flujo de datos persistentes: Gestionar información que sobrevive al cierre del navegador.
- Preparar el terreno para APIs: Dejar la lógica lista para reemplazar el localStorage por una base de datos real en el futuro.

5. RÚBRICA DE EVALUACIÓN

Criterio	Excelente (100%)	Aceptable (70%)	Insuficiente (0-40%)
Funcionalidad	Registro y Login validan datos correctamente en localStorage.	El sistema funciona pero permite registros duplicados.	No hay validación real de datos o no persiste la sesión.
Protección	Las rutas de admin están totalmente bloqueadas para usuarios sin permiso.	La protección funciona pero se puede evadir recargando la página.	Cualquier usuario puede entrar a cualquier URL sin loguearse.
TypeScript	Se utilizan interfaces y tipos específicos en todo el proyecto.	Uso excesivo de any o tipos mal definidos.	El código no aprovecha las ventajas de TypeScript.

Entrega	Entrega archivo .zip correctamente comprimido y con estructura de carpetas respetada.	Entrega archivo .zip funcional pero con detalles menores.	No entrega archivo .zip, está corrupto, no respeta la estructura solicitada o el proyecto no ejecuta.